

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\`a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-aa5af12ac6a452ea4.jsonl`  
- SHA-256: `4a49b977d25b9961234637fa98f70a241449955491bb9482dfed6edb5860756b`  
- Source modified: `2026-05-31T04:19:17+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T04:18:40.839Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMV3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:

****URL:**** <https://reducto.ai/compare/reducto-vs-llamaparse>

****Title:**** Reducto vs LlamaParse: Financial Document Processing Comparison

****Found via:**** Approval-gated paid/cloud fallbacks & data terms (cost/tradeoffs) search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:18:43.762Z)

I'll fetch the page content using WebFetch. Let me first load its schema.

3. assistant (2026-05-31T04:18:46.657Z)

I'll fetch the page content now.

4. user (2026-05-31T04:18:56.950Z)

Reducto vs LlamaParse: Key Factual Claims

Accuracy & Benchmarks

****Reducto table extraction:****

- "0.90 table similarity score on RD-TableBench"
- Handles "merged cells, multi-level headers, rotated text, and tables with missing borders"
- Claims "Up to 99-100% accuracy on complex real-world documents in a zero-shot setting"

****LlamaParse table extraction:****

- "Solid performance on simple tables. Performance drops on dense, structured tables"
- Quality described as "iffy on complex documents"

Complex Document Handling

Reducto: "Specifically designed to handle multi-column layouts, mixed-content pages, figures, charts, and scanned documents with high accuracy across the board"

LlamaParse: "Documents with irregular structure, overlapping content, or heavy table use are a noted weakness"

Data Handling & Compliance

Reducto: "SOC 2 Type II, HIPAA compliant (BAA available). Zero data retention on Growth tier and above. VPC, on-premises, and air-gapped deployment options available"

LlamaParse: "No documented enterprise compliance features, SOC 2, or flexible deployment options"

Pricing

Reducto: "Pay-as-you-go from \$0.015/page with 15,000 free credits to start"

LlamaParse: Approximately "40% lower than Reducto for basic parsing"

Table/Structured Output Features

Reducto: "Extract API with per-field citations, bounding boxes, and confidence scores" and "Every extracted field links to its exact bounding-box position"

LlamaParse: "Structured output was deprecated from LlamaParse in favor of LlamaExtract, a separate product"

Publication Date

"Last updated: May 20, 2026"

5. assistant (2026-05-31T04:19:02.322Z)

This is a vendor comparison page published by Reducto comparing itself against a competitor (LlamaParse). I have enough to assess source quality and extract claims. The key analytical note: this is first-party marketing content from one of the two compared products, so it is inherently biased toward Reducto ? relevant for the research question's "approval-gated cloud fallback" section, but the self-favoring benchmark and "iffy" competitor characterizations must be treated as vendor claims, not neutral measurement.

Let me extract the falsifiable, research-relevant claims.

6. user (2026-05-31T04:19:09.972Z)

Structured output provided successfully

7. assistant (2026-05-31T04:19:17.748Z)

Done. Source assessed and claims extracted.

Source quality: blog (vendor marketing). This is a first-party comparison page published by Reducto itself, comparing Reducto against competitor LlamaParse ? inherently biased toward Reducto. The self-reported "0.90 on RD-TableBench" benchmark is notable because RD-TableBench is also a Reducto-created benchmark, and the competitor characterizations ("iffy," "performance drops") are vendor framing, not neutral third-party measurement. I flagged the central data-handling claims as the most useful for muneem's approval-gated cloud-fallback section, since they bear directly on whether a cloud service could ever touch sensitive statement data (zero data retention, on-prem/air-gapped, BAA).

Key relevance to the research question: both products are cloud SaaS, so for muneem's hard local-only constraint they belong only in the approval-gated fallback tier. Reducto's air-gapped/on-prem and zero-retention options are the one detail that could matter if cloud is ever approved; LlamaParse's lack of documented compliance/self-hosting makes it weaker for sensitive financial data ? though that assessment comes from Reducto's own page and should be independently verified.